

SOLID Design Principles

Part 2



LISKOV SUBSTITUTION PRINCIPLE

If It Looks Like A Duck, Quacks Like A Duck, But Needs Batteries - You Probably Have The Wrong Abstraction

Liskov Substitution Principle (LSP)

(child class)

***Subtypes must be substitutable
for their base types***

(parent class)

Derived classes must be substitutable for their
base classes

Liskov Substitution Principle

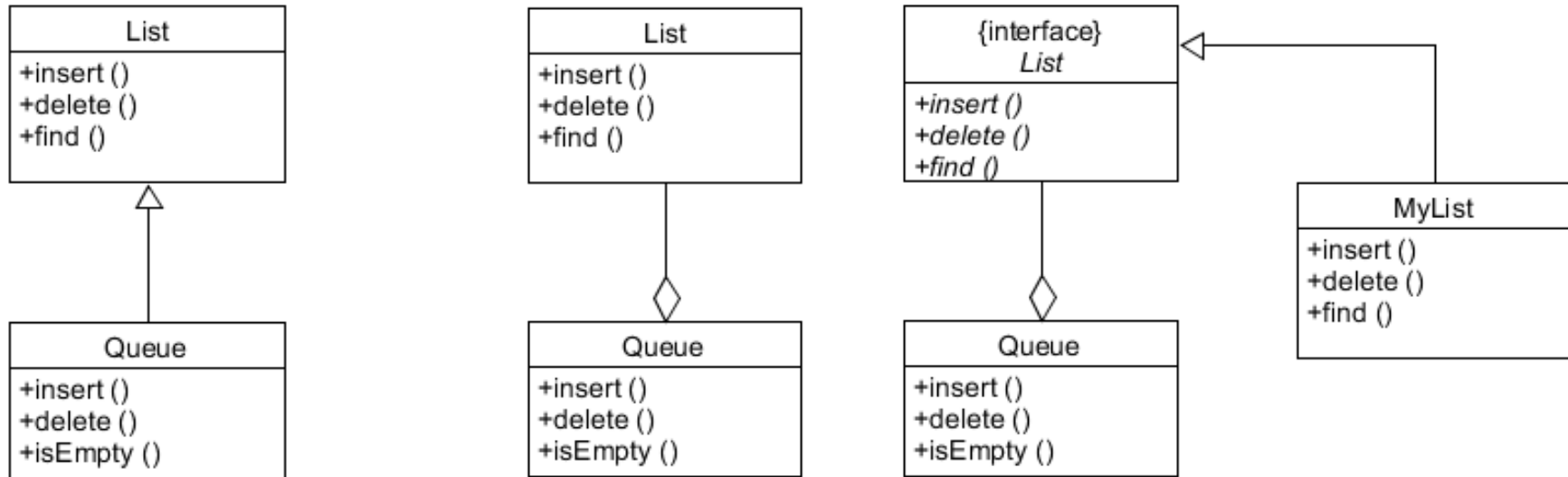
- A rule that you want to **check when you decide to use inheritance or not**
- If **C is a subtype of P** , then objects of type P may be replaced with objects of type C **without altering** any of the desirable properties of the program

Subtyping VS Implementation Inheritance

- Subtyping
 - establishes an IS_A relationship
 - also known as *interface inheritance*
- Implementation Inheritance
 - only reuses implementation and establishes a syntactic relationship not necessarily a semantic relationship
 - Also known as *code inheritance*
- Most OOP languages like Java, C++, and C#, inheritance keyword such as “extends” does the both Subtyping and Implementation Inheritance
 - But some languages distinguish them

The Liskov Substitution Principle and Reuse

- Think twice when you decide to use Inheritance!
 - If you want to reuse implementation of List, you had better exploit **object composition**, **not inheritance**.
 - If you inherit Queue from List, then you violate LSP since Queue object cannot be substitutable for List.



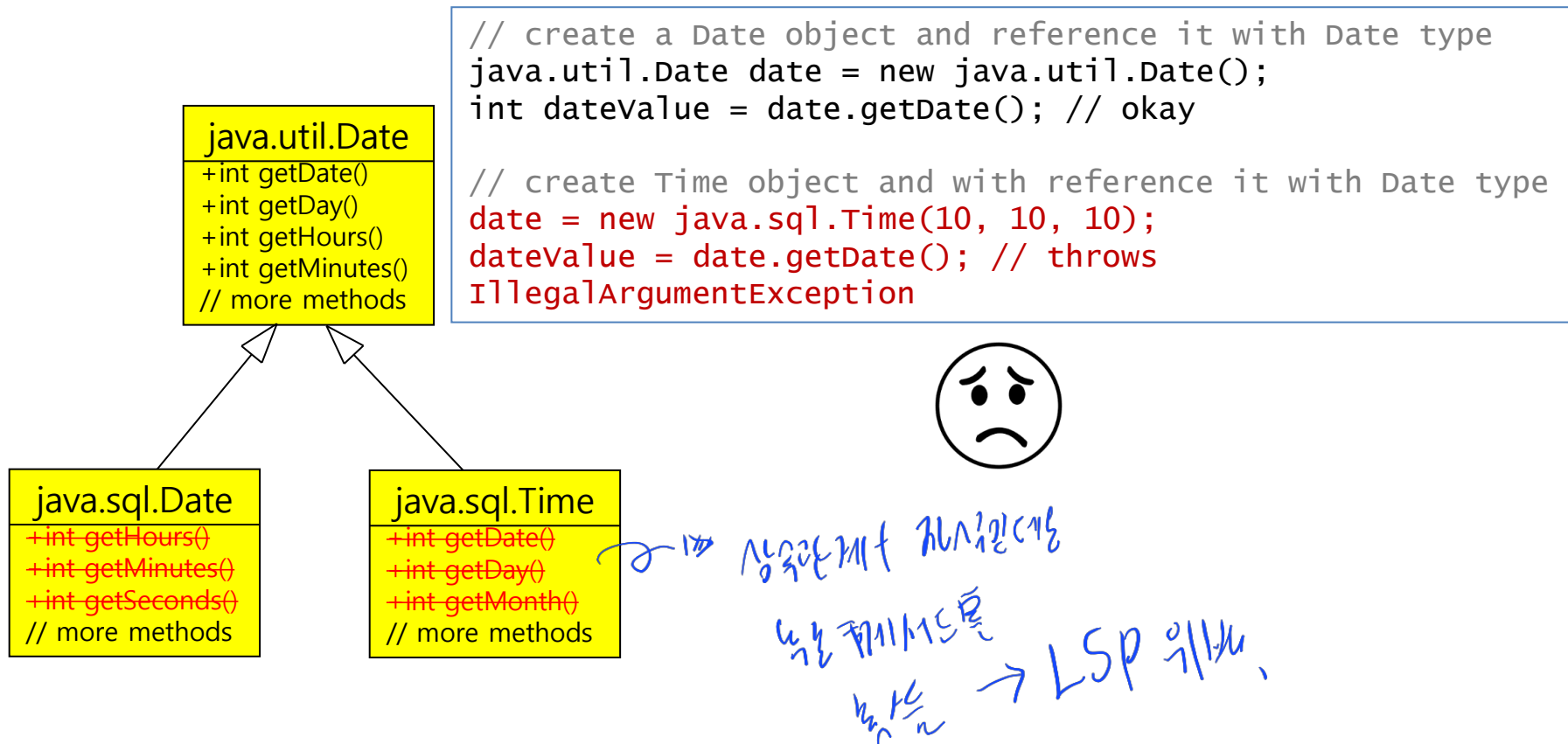
ex> list의 insert, delete는 임의의 곳에서 가능, but Queue는 FIFO.
즉, Queue가 list 일부 기능을 못함

Violation of LSP!

Good

Better!

Violation Example of LSP

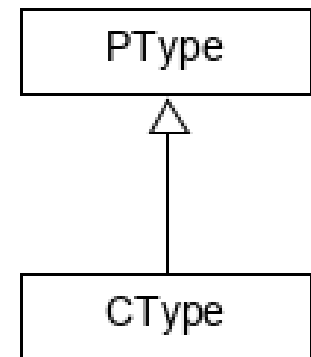


Violation of LSP may lead to another violation

- Assume that when *CType* is passed to *f()* instead of *PType*, it causes *f* to misbehave; **it means that *CType* violates the LSP**

→ *CType* is fragile in the presence of *f*

```
void f(PType x) {  
    // some code misbehevaves when x is CType  
    ...  
}
```



The owner of *f* might want to put **test code for *Ctype***

```
void f(PType x) {  
    if (x instanceof( CType)) throw new RuntimeException();  
    // some code misbehevaves when x is CType  
    ...  
}
```

The above reaction is worse. Now, it is violating OCP, too!

→ *f* is **not closed** to all various **subtypes of *PType***



Design by Contract

- Advertised Behavior of an object:
 - advertised **Requirements** (Preconditions)
 - advertised **Promises** (Postconditions)

When redefining a method in a derivate class, you may only replace its precondition by a weaker one, and its postcondition by a stronger one

B. Meyer, 1988

⇒ Derived class services should **require no more** and **promise no less**

requirement -> 약화만 가능

promise -> 강화만 가능

자식이 부모가 했던거 보다 덜 제공하거나 더 많이 요구하는건 불가능



DEPENDENCY INVERSION PRINCIPLE

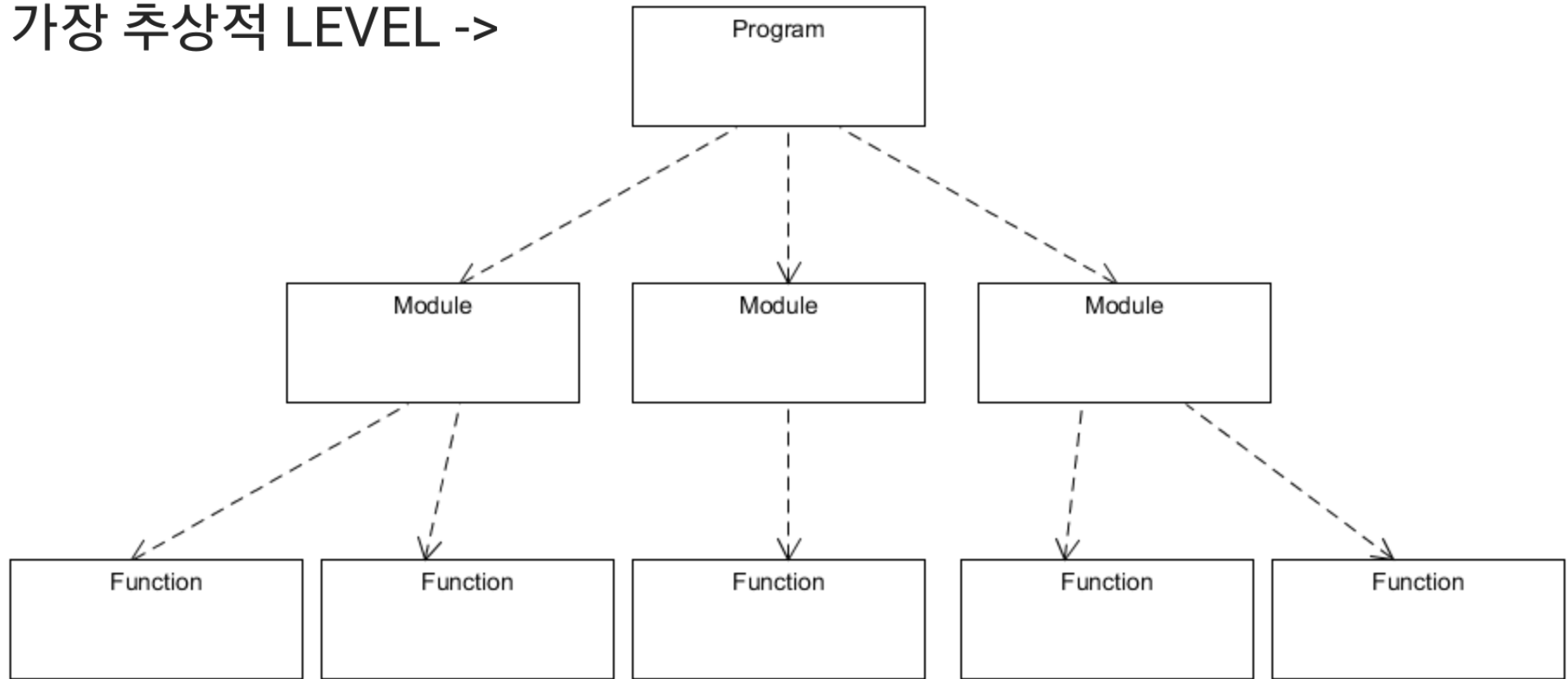
Would You Solder A Lamp Directly To The Electrical Wiring In A Wall?

Dependency Inversion Principle (DIP)

- *High-level modules should not depend on low-level modules. Both should depend on abstractions.*
- *Abstractions should not depend on details. Details should depend on abstractions.*
- Why Inversion?
 - DIP attempts to “invert” the dependencies that result from a structured analysis and design approach

Typical in Structured Analysis & Design

가장 추상적 LEVEL ->



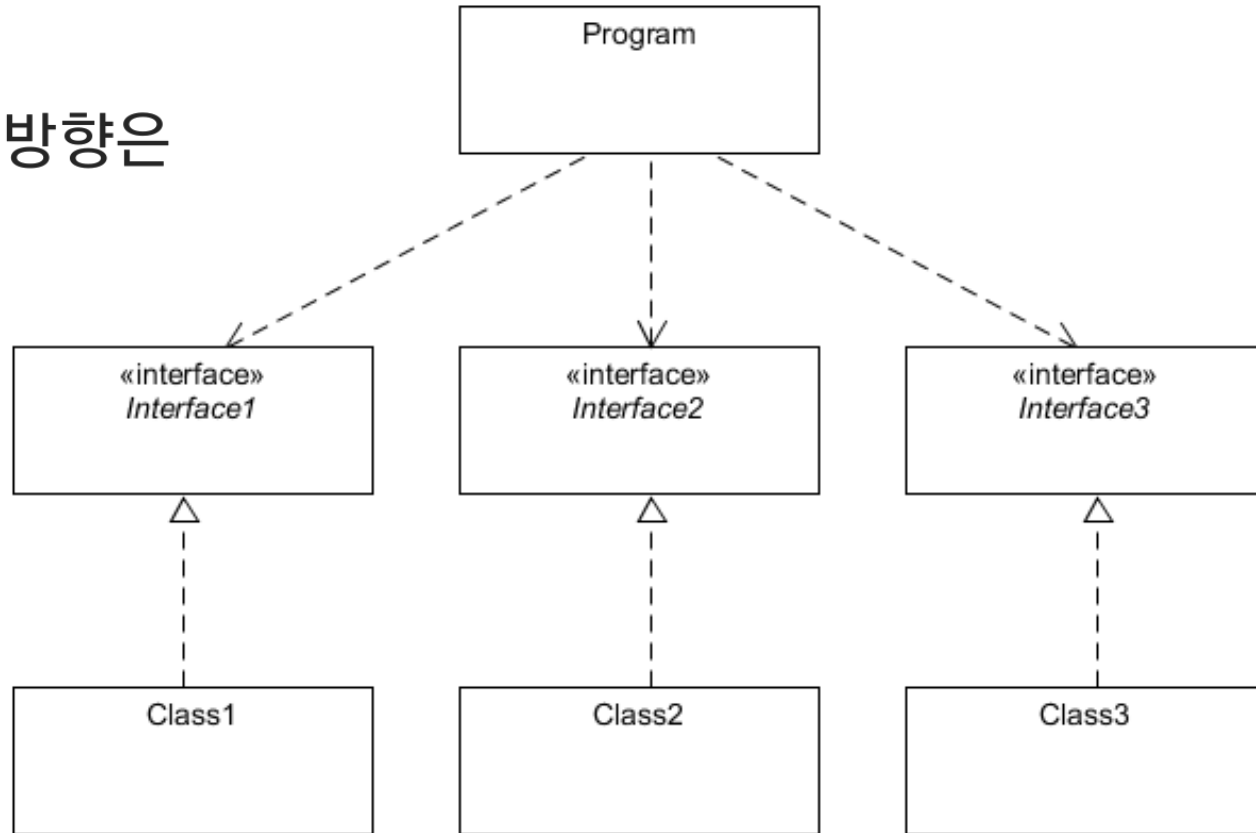
<- 가장 구체적 LEVEL

=> function이 바뀌면 program까지 영향을 끼칠수 있음

Dependency Inversion Principle

Interfaces and abstract classes are high-level resources
Concrete classes are low-level resources

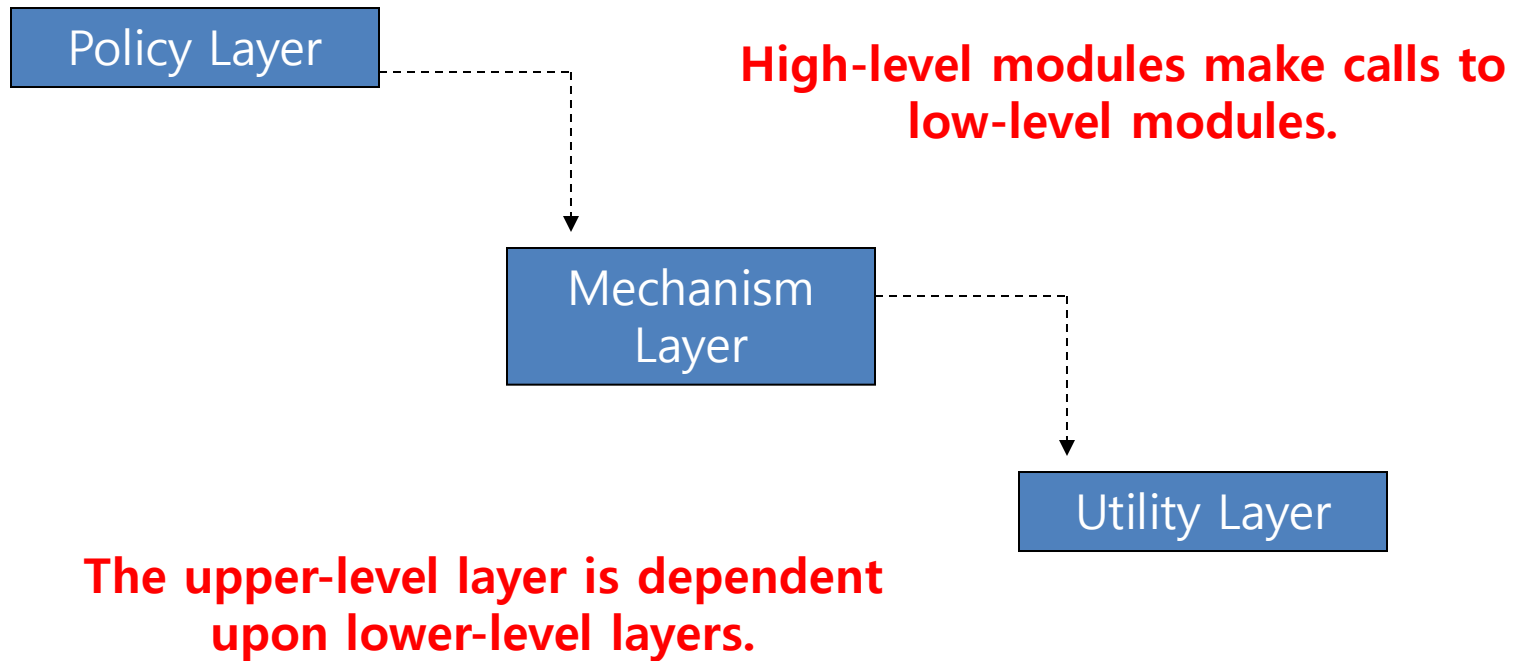
단, control 방향은
여전히
p-> c로



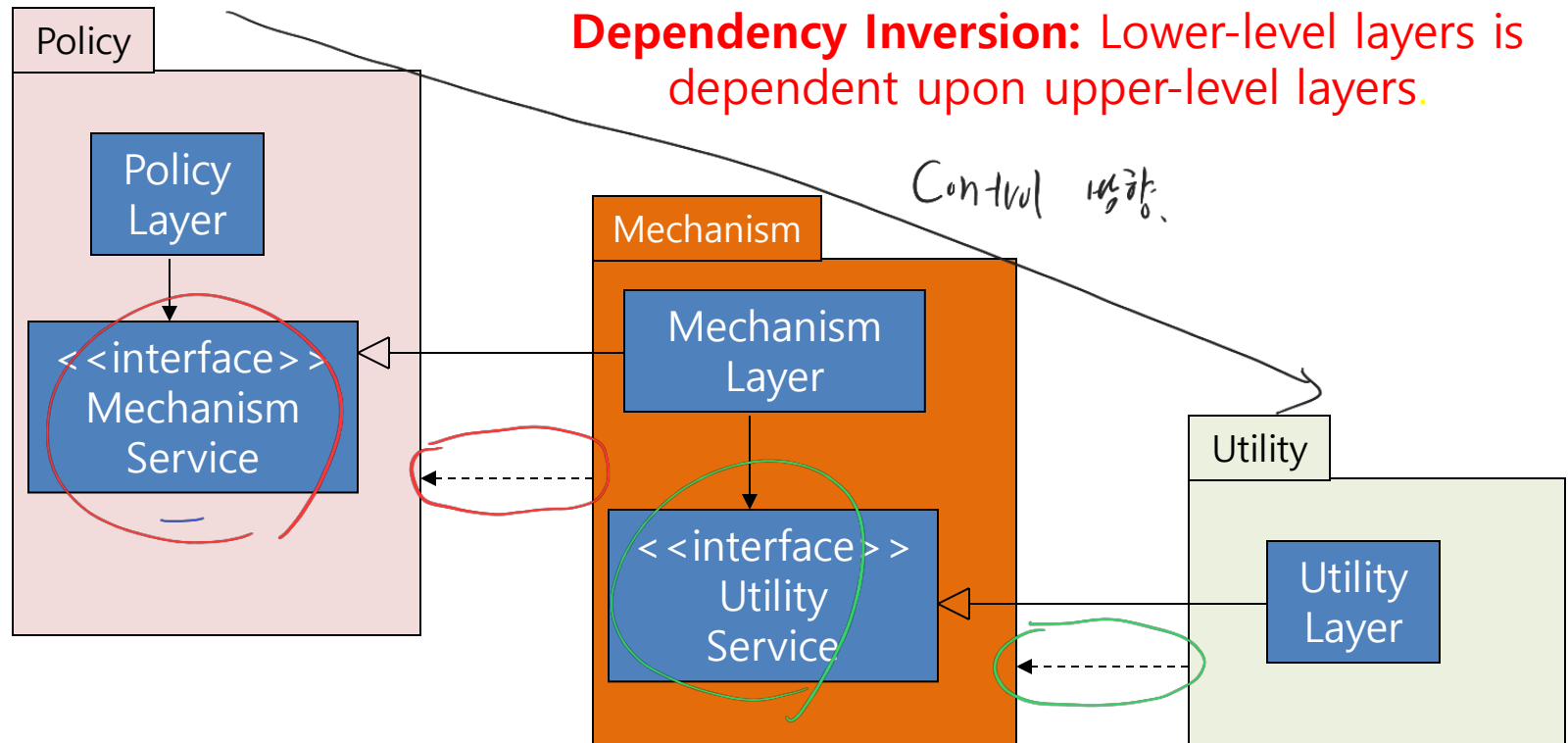
Inversion of Ownership

- Its not just an inversion of **dependency**, DIP also inverts **ownership**
 - Typically a service interface is “owned” or declared by the server, here the client is specifying what they want from the server
 - DIP → Clients should own the interface!

The dependency inversion principle



The dependency inversion principle



The client (upper-level layer) owns the interface, not the lower-level layers

↳ 상위 layer가 하위 layer를 지배하는 방향으로 변경.



INTERFACE SEGREGATION PRINCIPLE

You Want Me To Plug This In, Where?

Interface Segregation Principle (ISP)

Clients should not be forced to depend on methods they do not use

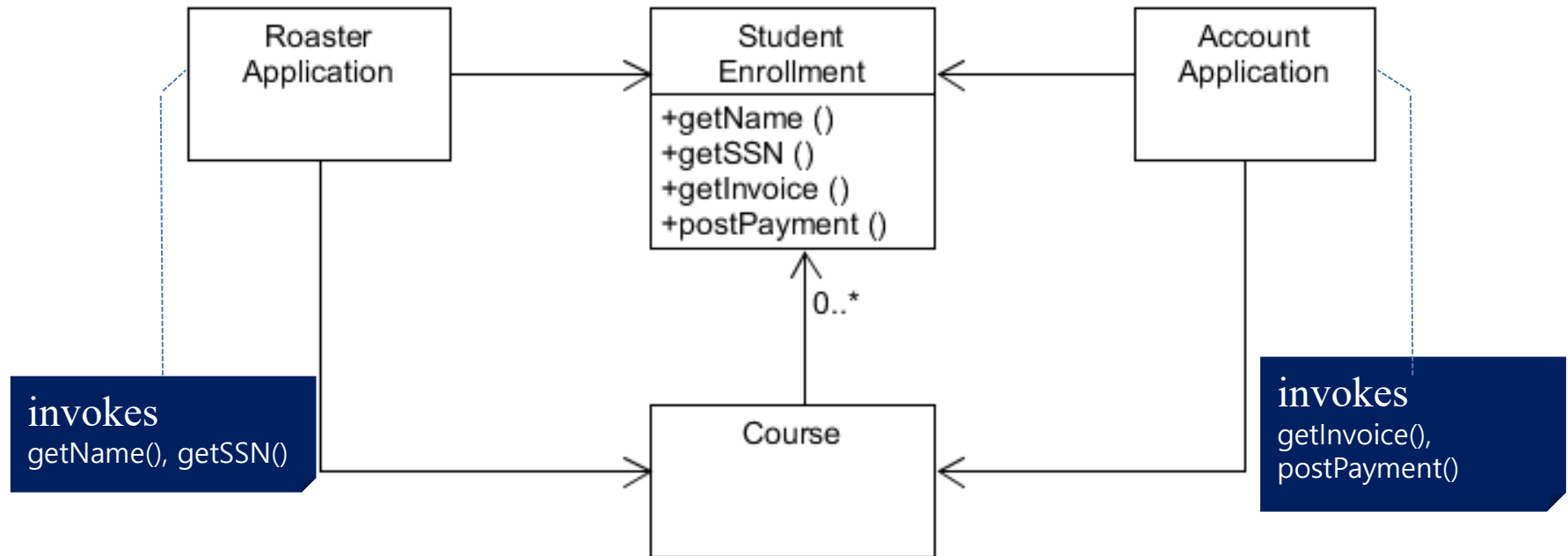
Make fine grained interfaces that are client specific

-> 인터페이스를 더 세분화 시켜라

Fat interface

- Bundling functions for different clients into one interface create unnecessary coupling among the clients.
 - When one client causes the interface to change, all other clients are forced to recompile.
 - **Solution: Break the interface into cohesive groups**
- ISP solves non-cohesive interfaces
 - Clients should know only abstract base classes that have cohesive interfaces

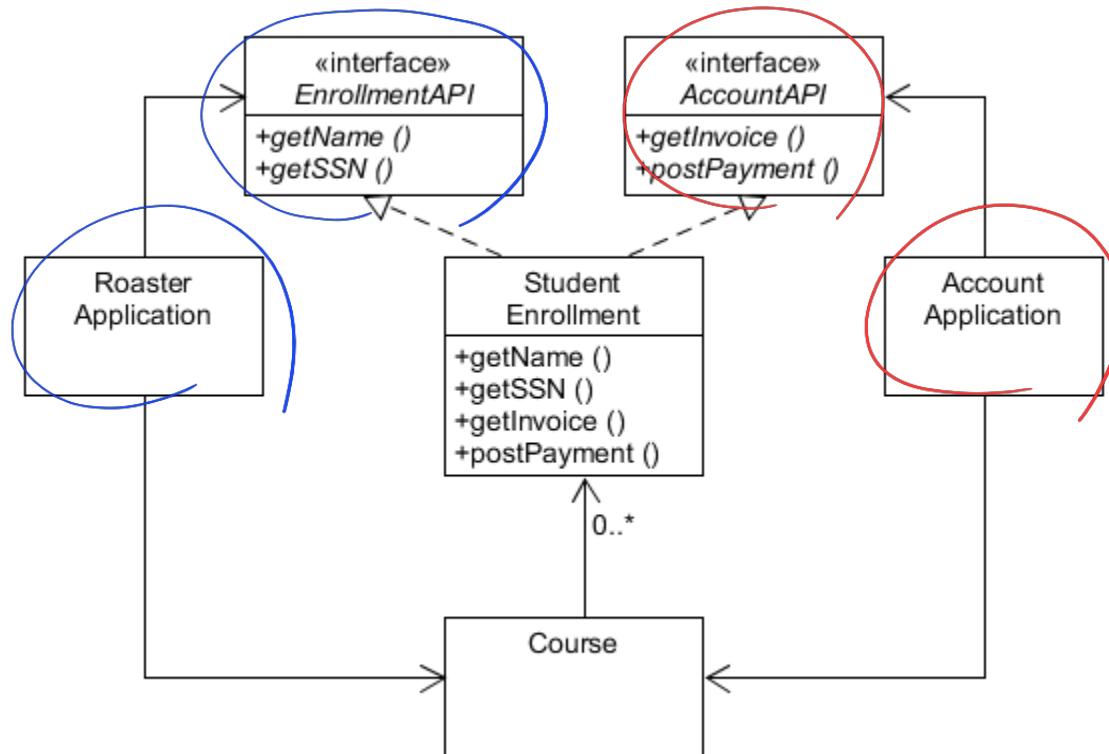
ISP Example: Original Design



- Suppose that **RoasterApplication** does not invoke methods `getInvoice()` or `postPayment()`
- Also suppose **AccountApplication** does not invoke the methods `getName()` or `getSSN()`
- **Requirements change: add a new argument to the `postPayment()`**
- This change force us to **recompile and redeploy **RoasterApplication****, which does not care at all about the `postPayment()`

ISP Example: Better Design

interface -> cohesion 높음



- Now, each user of a `Student Enrollment` object is given an interface that provides just the methods that it is interested in
- This protects the user from changes in methods that don't concern it
- It also protects the user from knowing too much about the implementation of the object it is using.

Summary

■ SOLID

- Single-Responsibility Principle
- Open-Closed Principle
- Liskov Substitution Principle
- Dependency Inversion Principle
- Interface Segregation Principle

~~SRP~~ ↔ ISP

↳ 둘 다 지키는 게 X.

■ Design Principles

- Help manage dependency
- Better maintainability, flexibility, robustness, and reusability
- Abstraction is important
- GRASP patterns/principles